=== UNIT 6: FILE I/O, SERIALIZATION & EXCEPTION HANDLING ===

>> 6.1 File Streams and File Access Modes

Persistent data storage requires transferring data between volatile RAM and non-volatile disk files:
* The open() Built-in Function: file_obj = open(filename, mode='r', encoding='utf-8').
* Standard File Modes: 'r' (read only), 'w' (write only, truncates existing file), 'a' (append), 'r+' (read/write), 'b' (binary mode for images/compiled data).
* Context Managers and the 'with' Statement: Encapsulates file handling within an automated resource management block based on the __enter__ and __exit__ dunder protocols. Guarantees that the file descriptor is closed automatically, even if unhandled exceptions occur.
* Reading Methods: .read(size) reads entire file or up to size characters; .readline() reads one line; .readlines() reads all lines into a list of strings.
* Writing Methods: .write(string) writes text; .writelines(list_of_strings) writes sequences.

>> 6.2 Data Serialization: CSV and JSON

* JSON (JavaScript Object Notation): Universal lightweight data interchange format supported by built-in json module: json.dumps(obj) (serialize to string), json.dump(obj, file) (serialize to file), json.loads(str) (parse from string), json.load(file) (parse from file).
* CSV (Comma-Separated Values): Managed via csv.reader, csv.writer, csv.DictReader, and csv.DictWriter.

>> 6.3 Exception Architecture and Handling Hierarchy

An exception is a runtime error event that disrupts the normal programmatic instruction flow:
* Exception Hierarchy: All exceptions inherit from BaseException. Standard programmatic errors derive from Exception: (ArithmeticError -> ZeroDivisionError, LookupError -> IndexError, KeyError, ValueError, TypeError, FileNotFoundError).
* The Complete Try-Except Block Structure:
- try: Critical code block monitored for potential exceptions.
- except SpecificException as err: Handles matched exception type.

- else: Executes ONLY if the try block completes successfully WITHOUT exceptions.
- finally: Cleanup block guaranteed to execute unconditionally (e.g. closing database connections).

>> 6.4 Raising Exceptions and Custom Exception Classes

* The raise Statement: Explicitly triggers an exception: raise ValueError('Invalid input').
* Custom User-Defined Exceptions: Created by subclassing the standard Exception class:
class InsufficientBalanceError(Exception): pass.

[PROGRAM 1: Safe File Processor with 'with' and Exception Handling]

Problem Context: Write a program reading a configuration file containing key=value pairs, parsing valid entries into a dictionary, handling FileNotFoundError and parsing errors cleanly.
Python Source Code:

```
def parse_config(file_path):
config = {}
try:
with open(file_path, 'r', encoding='utf-8') as f:
for line_no, line in enumerate(f, start=1):
clean_line = line.strip()
if not clean_line or clean_line.startswith('#'):
continue
if '=' not in clean_line:
print(f'Warning: Line {line_no} malformed, skipping.')
continue
key, val = clean_line.split('=', 1)
config[key.strip()] = val.strip()
except FileNotFoundError:
print(f'Error: File '{file_path}' not found on disk.')
return None
except PermissionError:
print(f'Error: Insufficient permission to read '{file_path}'.')
return None
else:
print(f'Successfully parsed {len(config)} config parameters.')
```

return config

# Test with non-existent file
parse_config('missing_config.env')
Execution Trace & Output:
Output: 'Error: File 'missing_config.env' not found on disk.' Application terminates gracefully without crashing.

[PROGRAM 2: JSON Data Serialization and Deserialization Pipeline]

Problem Context: Model a student record database, serialize the dictionary into an encrypted/formatted JSON file on disk, read it back, and verify data parity.
Python Source Code:

```
import json

students_db = [
{'id': 101, 'name': 'Aarav', 'courses': ['CSE101', 'MTH165'], 'gpa': 3.85}
{'id': 102, 'name': 'Diya',  'courses': ['INT108', 'GEN231'], 'gpa': 3.92}
]

json_path = 'students.json'
# Serialization to file
with open(json_path, 'w', encoding='utf-8') as f:
json.dump(students_db, f, indent=4)
print('JSON data serialized to disk.')

# Deserialization from file
with open(json_path, 'r', encoding='utf-8') as f:
loaded_data = json.load(f)
print(f'Loaded {len(loaded_data)} records. First student: {loaded_data[0]['name']}')
```

Execution Trace & Output:
JSON file written with 4-space indentation; deserialized loaded_data matches original students_db list.

[PROGRAM 3: Custom Business Logic Exception Classes]

Problem Context: Implement a shopping cart checkout module with a custom InsufficientStockError exception raised when inventory is exceeded.

Python Source Code:

```
class InsufficientStockError(Exception):
def __init__(self, item, requested, available):
super().__init__(f'Cannot purchase {requested} units of '{item}'. Only {available}
available.')
self.item = item
self.requested = requested
self.available = available

inventory = {'Laptop': 5, 'Mouse': 20}

def purchase(item, quantity):
if item not in inventory:
raise KeyError(f'Item '{item}' does not exist in inventory.')
if quantity > inventory[item]:
raise InsufficientStockError(item, quantity, inventory[item])
inventory[item] -= quantity
print(f'Purchase successful! Remaining {item} stock: {inventory[item]}')

try:
purchase('Laptop', 8)
except InsufficientStockError as err:
print(f'Checkout Failed: {err}')
```

Execution Trace & Output:

Output: 'Checkout Failed: Cannot purchase 8 units of 'Laptop'. Only 5 available.'

[PROGRAM 4: CSV Data Parsing and Summary Statistics]

Problem Context: Write a program generating a CSV sales file, read it using csv.DictReader, and compute total revenues and sales by category.

Python Source Code:

```
import csv

csv_file = 'sales.csv'
sales_data = [
['TransactionID', 'Category', 'Amount']
['TX101', 'Electronics', '12500']
['TX102', 'Stationery',  '450']
['TX103', 'Electronics', '8500']
['TX104', 'Books',       '1200']
```

]

with open(csv_file, 'w', newline='', encoding='utf-8') as f:
writer = csv.writer(f)
writer.writerows(sales_data)

category_revenue = {}
with open(csv_file, 'r', encoding='utf-8') as f:
reader = csv.DictReader(f)
for row in reader:
cat = row['Category']
amt = float(row['Amount'])
category_revenue[cat] = category_revenue.get(cat, 0.0) + amt

for cat, total in category_revenue.items():
print(f'Category {cat:12s} : INR {total:10,.2f}')
Execution Trace & Output:
Electronics: INR 21,000.00 | Stationery: INR 450.00 | Books: INR 1,200.00.

>> 6.5 University Review & Examination Questions

* Q1: Explain the purpose and operation of Context Managers ('with' statement) in Python file I/O.
Answer: The 'with' statement simplifies resource management by guaranteeing that setup and teardown actions occur reliably around a block of code. For file I/O, 'with open(...) as f:' binds the opened stream to f. Internally, the context manager calls the __enter__() method when entering the block, and guarantees that __exit__() is executed upon leaving the block (even if a critical exception occurs, an unhandled error is raised, or a return statement is executed). This prevents file descriptor leaks, operating system resource exhaustion, and file corruption caused by unclosed buffers.
* Q2: Describe the role of 'else' and 'finally' blocks in Python exception handling.
Answer: In a try-except structure: The 'except' block catches and handles designated exceptions that occurred during try execution. The 'else' block executes ONLY if the try block completes successfully with ZERO exceptions raised. It isolates code that should not be accidentally monitored by the try handlers. The 'finally' block executes unconditionally in all circumstances: whether an exception was raised, caught, re-raised, or if no exception occurred. It is reserved for essential resource cleanup, such as closing network sockets, database connections, or freeing hardware locks.

>> 6.6 Key Takeaways and Revision Points

* The 'with' statement guarantees automated file closing via context management.
* json.dump() and json.load() handle structured data serialization to disk.
* 'try-except-else-finally' cleanly separates error detection, normal flow, and cleanup.
* Custom exceptions inherit from the built-in Exception class.